COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/09/2026	**DATE SUBMITTED:**	24/09/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 9.1 TREE ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

TASK 1:

Header	main

tree.h

```h
#ifndef TREE_H
#define TREE_H
#include <iostream>

struct Node {
    char data;
    Node* firstChild;
    Node* nextSibling;

    Node(char value) {
        data = value;
        firstChild = nullptr;
        nextSibling = nullptr;
    }
};
// Add a child to a parent node
void addChild(Node* parent, Node* child) {
    if (parent->firstChild == nullptr) {
        parent->firstChild = child;
    }
    else {
        Node* temp = parent->firstChild;

        while (temp->nextSibling != nullptr) {
            temp = temp->nextSibling;
        }

        temp->nextSibling = child;
    }
}
// Display the tree
void displayTree(Node* root, int level = 0) {
    if (root == nullptr) {
        return;
    }

    for (int i = 0; i < level; i++) {
        std::cout << "  ";
    }

    std::cout << root->data << std::endl;

    Node* child = root->firstChild;

    while (child != nullptr) {
        displayTree(child, level + 1);
        child = child->nextSibling;
    }
}

#endif
```

task1_main.cpp

```cpp
#include <iostream>
#include "tree.h"

int main() {

    // Create the nodes
    Node* A = new Node('A');
    Node* B = new Node('B');
    Node* C = new Node('C');
    Node* D = new Node('D');
    Node* E = new Node('E');
    Node* F = new Node('F');
    Node* G = new Node('G');
    Node* H = new Node('H');
    Node* I = new Node('I');
    Node* J = new Node('J');
    Node* K = new Node('K');
    Node* L = new Node('L');
    Node* M = new Node('M');
    Node* N = new Node('N');
    Node* P = new Node('P');
    Node* Q = new Node('Q');

    // Connect the nodes
    addChild(A, B);
    addChild(A, C);
    addChild(A, D);
    addChild(A, E);
    addChild(A, F);
    addChild(A, G);

    addChild(D, H);

    addChild(E, I);
    addChild(E, J);

    addChild(J, P);
    addChild(J, Q);

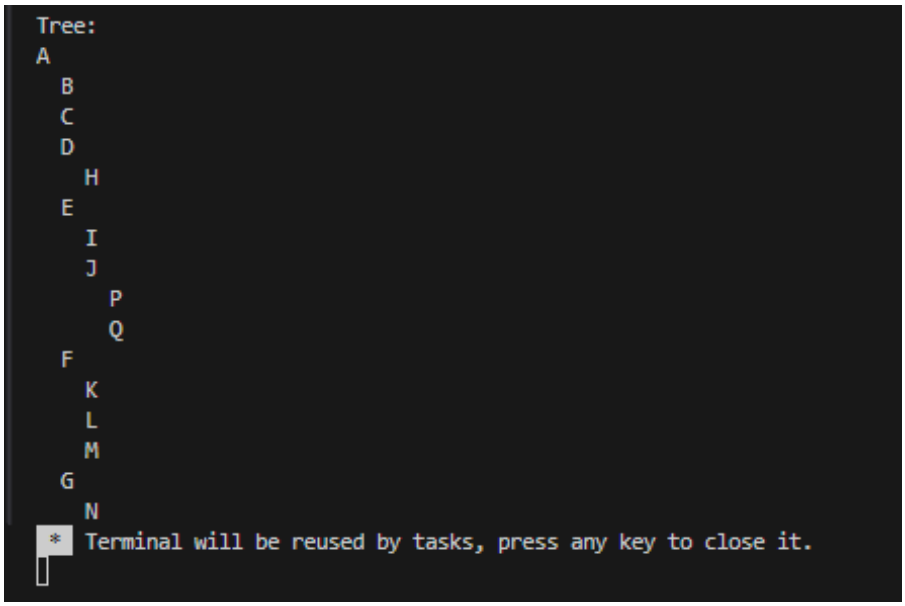
    addChild(F, K);
    addChild(F, L);
    addChild(F, M);

    addChild(G, N);

    // Display the tree
    std::cout << "Tree:" << std::endl;
    displayTree(A);

    return 0;
}
```

OUTPUT:



//ANALYSIS: The indentation of the output based on the depth so if the dept is 0 it has no spaces.

TASK 2:

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2

M	0	2
N	0	2
P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

TASK 3:

3.1

PRE-ORDER	`Pre-order: A B C D H E I J P Q F K L M G N`
POST-ORDER	`Post-order: B C H D I P Q J E K L M F N G A`
IN-ORDER	`In-order: B A C H D I E P J Q K F L M N G`

3.2

Header	Main

```cpp
// Pre-order
void preOrder(Node* root) {
    if (root == nullptr) {
        return;
    }

    std::cout << root->data << " ";

    Node* child = root->firstChild;

    while (child != nullptr) {
        preOrder(child);
        child = child->nextSibling;
    }
}

// Post-order
void postOrder(Node* root) {
    if (root == nullptr) {
        return;
    }

    Node* child = root->firstChild;

    while (child != nullptr) {
        postOrder(child);
        child = child->nextSibling;
    }

    std::cout << root->data << " ";
}

// In-order
void inOrder(Node* root) {
    if (root == nullptr) {
        return;
    }

    Node* child = root->firstChild;

    if (child != nullptr) {
        inOrder(child);

        std::cout << root->data << " ";

        child = child->nextSibling;

        while (child != nullptr) {
            inOrder(child);
            child = child->nextSibling;
        }
    }
    else {
        std::cout << root->data << " ";
    }
}

#endif
```

```cpp
    // Traversals
    std::cout << "\nPre-order: ";
    preOrder(A);
    std::cout << std::endl;

    std::cout << "Post-order: ";
    postOrder(A);
    std::cout << std::endl;

    std::cout << "In-order: ";
    inOrder(A);
    std::cout << std::endl;

```

//In the main file I only add these functions to print out the output with the same header.

OUTPUT:

```
Pre-order: A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order: B A C H D I E P J Q K F L M N G
```

3.3

Header	Main

```cpp
bool found;
// Search Pre-order
void searchPreOrder(Node* root, char key) {
    if (root == nullptr || found) {
        return;
    }

    if (root->data == key) {
        found = true;
        return;
    }

    Node* child = root->firstChild;

    while (child != nullptr) {
        searchPreOrder(child, key);
        child = child->nextSibling;
    }
}

// Search Post-order
void searchPostOrder(Node* root, char key) {
    if (root == nullptr || found) {
        return;
    }

    Node* child = root->firstChild;

    while (child != nullptr) {
        searchPostOrder(child, key);
        child = child->nextSibling;
    }

    if (root->data == key) {
        found = true;
    }
}

// Search In-order
void searchInOrder(Node* root, char key) {
    if (root == nullptr || found) {
        return;
    }

    Node* child = root->firstChild;

    if (child != nullptr) {
        searchInOrder(child, key);

        if (root->data == key) {
            found = true;
            return;
        }

        child = child->nextSibling;

        while (child != nullptr && !found) {
            searchInOrder(child, key);
            child = child->nextSibling;
        }
    }
    else {
        if (root->data == key) {
            found = true;
        }
    }
}
```

```cpp
    std::cout << "\nFind Data:" << std::endl;
    findData(A, 1, 'J');  // Pre-order
    findData(A, 2, 'P');  // Post-order
    findData(A, 3, 'G');  // In-order

```

```
178  void findData(Node* root, int choice, char key) {
179      found = false;
180
181      if (choice == 1) {
182          searchPreOrder(root, key);
183      }
184      else if (choice == 2) {
185          searchPostOrder(root, key);
186      }
187      else if (choice == 3) {
188          searchInOrder(root, key);
189      }
190
191      if (found) {
192          std::cout << key << " was found!" << std::endl;
193      }
194  }
```

OUTPUT:

```
Find Data:
J was found!
P was found!
G was found!
```

3.4

main	Output

```
68      // TASK 3.4 new leaf node:
69      Node* O = new Node('O');
70      addChild(G, O);
71
72      // Test findData after adding O
73      std::cout << "\nAfter adding O:" << std::endl;
74
75      findData(A, 1, 'O');
76      findData(A, 2, 'O');
77      findData(A, 3, 'O');
78
79      return 0;
```

```
After adding O:
O was found!
O was found!
O was found!
```

CONCLUSION / CODE ANALYSIS:

This procedure, we did the tree by creating a node and connecting them. Using traversal functions to find the pre-order, in-order, and post-order of the tree, we were able to see the different orders of the nodes in the output. The first procedure task output is based on depth in the table so the 0,1, and 2 are the spaces indentation. For task 3 we also used the findData() function to search a specific node and print out when it's found.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1:

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

Main

supplementary.cpp > main()

```cpp
#include <iostream>
#include "BST.h"

int main(){

    BinarySearchTree<int>* tree = new BinarySearchTree(2);

    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);

    std::cout<<"Pre-order Output: ";
    tree->preOrder();
    std::cout<<std::endl;
    std::cout<<"In-order Output: ";
    tree->inOrder();
    std::cout<<std::endl;
    std::cout<<"Post-order Output: ";
    tree->postOrder();
    std::cout<<std::endl;


    return 0;
}
```

Header

tree.h | BST.h

BST.h > ...

```cpp
#ifndef BST_H
#define BST_H


template <typename T>
class BinarySearchTree{
    private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;
    public:
    //constructor:
    BinarySearchTree(T value);
    // insertion:
    void insert(T value);

    // TRAVERSAL:
    void preOrder();
    void inOrder();
    void postOrder();

};

template<typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

template<typename T>
void BinarySearchTree<T>::insert (T value){
    // go to left if: value < key
    if(value < key){

        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }else{
            left->insert(value);
        }
    }
    else if(value > key){
        if (right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right->insert(value);
        }
    }
}
// ============ PRE-ORDER ===============
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<< key << " ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
// ============ IN-ORDER ===============
template<typename T>
void BinarySearchTree<T>::inOrder(){

    if(left != nullptr){
        left->preOrder();
    }
     std::cout<< key << " ";

    if(right != nullptr){
        right->preOrder();
```

```cpp
// ============ IN-ORDER ================
template<typename T>
void BinarySearchTree<T>::inOrder(){

    if(left != nullptr){
        left->preOrder();
    }
     std::cout<< key << " ";

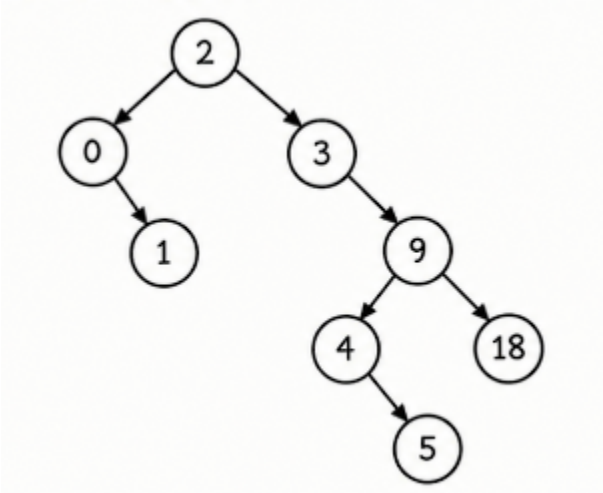
    if(right != nullptr){
        right->preOrder();
    }
}
// ============ POST-ORDER ================
template<typename T>
void BinarySearchTree<T>::postOrder(){

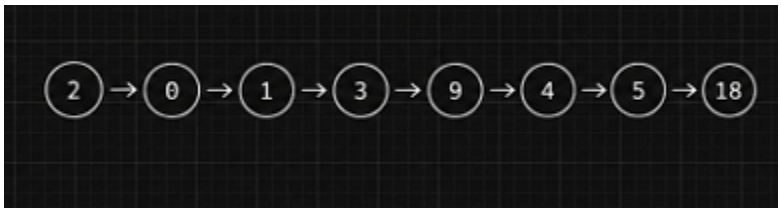
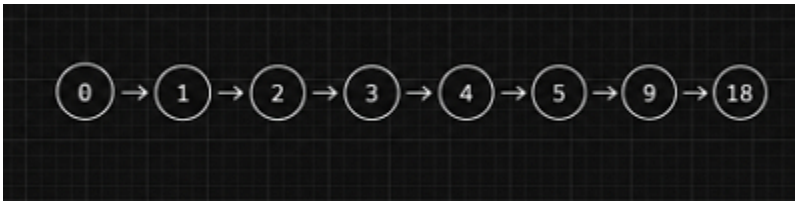
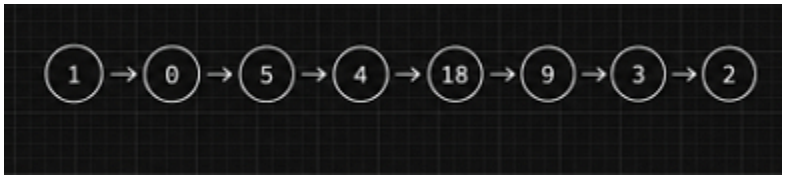
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }

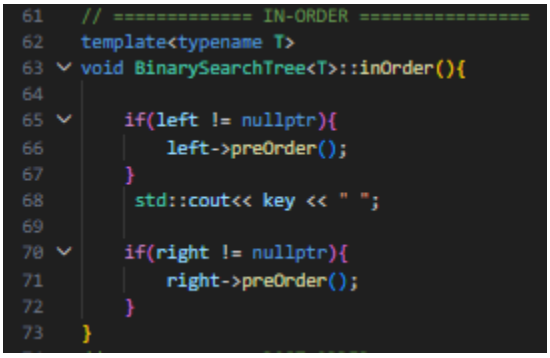
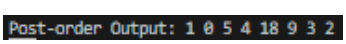
     std::cout<< key << " ";
}

#endif
```

Step 2:



Pre-order	
In-order	
Post-order	

Step 3:		
In-order	 In-order traversal was performed with the inorder() function, following the order left ⟶ root ⟶ right.	
Post-order		

```
61  // ============= IN-ORDER ================
62  template<typename T>
63  void BinarySearchTree<T>::inOrder(){
64
65      if(left != nullptr){
66          left->preOrder();
67      }
68       std::cout<< key << " ";
69
70      if(right != nullptr){
71          right->preOrder();
72      }
73  }
```

```
74  // ============= POST-ORDER ================
75  template<typename T>
76  void BinarySearchTree<T>::postOrder(){
77
78      if(left != nullptr){
79          left->postOrder();
80      }
81      if(right != nullptr){
82          right->postOrder();
83      }
84
85       std::cout<< key << " ";
86  }
```

```
In-order Output: 0 1 2 3 9 4 5 18
```

```
Post-order Output: 1 0 5 4 18 9 3 2
```

PRE-ORDER TRAVERSAL

HEADER:

```
51  // ============= PRE-ORDER ================
52  template<typename T>
53  void BinarySearchTree<T>::preOrder(){
54      std::cout<< key << " ";
55      if(left != nullptr){
56          left->preOrder();
57      }
58      if(right != nullptr){
59          right->preOrder();
60      }
61  }
```

MAIN:

```
17      std::cout<<"Pre-order Output: ";
18      tree->preOrder();
19      std::cout<<std::endl;
```

OUTPUT:

```
Pre-order Output: 2 0 1 3 9 4 5 18
```

POST-ORDER TRAVERSAL

HEADER:

```
75  // ============= POST-ORDER ================
76  template<typename T>
77  void BinarySearchTree<T>::postOrder(){
78
79      if(left != nullptr){
80          left->postOrder();
81      }
82      if(right != nullptr){
83          right->postOrder();
84      }
85
86      std::cout<< key << " ";
87  }
88
```

MAIN:

```
22      std::cout<<"Post-order Output: ";
23      tree->postOrder();
24      std::cout<<std::endl;
```

OUTPUT:

```
Post-order Output: 1 0 5 4 18 9 3 2
```

CONCLUSION:

In this supplementary activity, we implemented a binary search tree program that has the values of 2, 3, 9, 18, 0, 1, 4, 5 and prints out each traversal method. We also created a diagram for visualization of the tree. The in-order, pre-order, and post-order methods produced different sequences because, as you can see in the diagram in Step 2, each method follows a different way of visiting the nodes. I think I did well in this activity because I can now visualize what these methods do and how they work when traversing the tree but still need practice for this.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Ai Overview, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code concepts explanation and formatting**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for formatting each task and for concepts of the tree.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.